Data Structure

Haiyang Liu

Stack

- Stack as ADT
- Implementation of Stack
- Application of Stack

Stacks and queues

- Stacks and queues are linear lists where insertions and deletions are restricted to the "endpoints" of the list; in other words, they are linear lists with limited operations.

List	Stack	Queue
Insert(L, **i**, x)	Insert(S, **n+1**, x)	Insert(Q, **n+1**, x)
$1 \le i \le n+1$		
Delete(L, **i**)	Delete(S, **n**)	Delete(Q, **1**)
$1 \le i \le n$		

Type Definition of Stack

- **A linear list where insertions and deletions are restricted to the end of the list. The end (tail of the list) that allows insertions and deletions is called the top, and the other end (head of the list) is called the bottom.**
- **When there are no elements in the list, it is called an empty stack.**

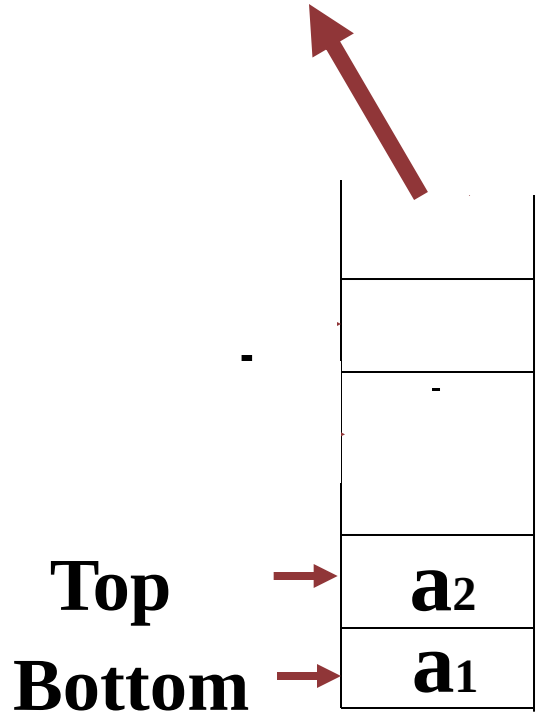
Stack

$S=(a_1,a_2,\ldots,a_n)$

Bottom Top

Push Order: $a_1, a_2, \ldots, a_n$

Pop Order: $a_n, a_{n-1}, \ldots, a_2, a_1$



A stack processes data elements according to the "Last In, First Out" principle, referred to as a LIFO list for short.

Definition of the Abstract Data Type for Stack

ADT Stack {

Data elements：

$$D = \{ a_i \mid a_i \in ElemSet,\ i=1,2,\ldots,n,\ \ n \geq 0 \}$$

Data relationship：

$$R_1 = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D,\ i=2,\ldots,n \}$$

// a_n as the top， a_1 as the bottom.

Basic Operations：

} ADT Stack

Basic Operations for Stack

- `InitStack(&S)`
- `DestroyStack(&S)`
- `StackLength(S)`
- `StackEmpty(s)`
- `GetTop(S, &e)`
- `ClearStack(&S)`
- `Push(&S, e)`
- `Pop(&S, &e)`
- `StackTravers(S, visit())`

Basic Operations (1)

InitStack(&S)

Initialization Operation

Result: **Construct an empty stack S.**

DestroyStack(&S)

Condition: **Stack S exists.**

Result: **Stack S destroyed**

Basic Operations (2)

StackLength(S)

Condition: Stack S exists.

Result: the number of elements in S, i.e., the length of the stack.

StackEmpty(S)

Condition: Stack S exists.

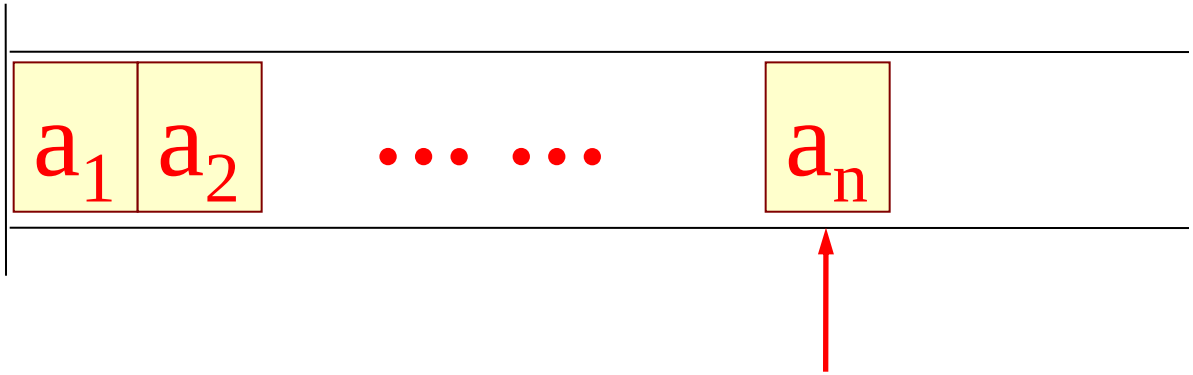
Result: If stack S is empty, return TRUE,
otherwise return FALSE。

Basic Operations (3)

GetTop(S, &e)

Condition: Stack S exists.

Result: Get the top element in Stack S, and store it in e.



Basic Operations (4)

ClearStack(&S)

Condition: Stack S exists

Result: Clear S to an empty stack.

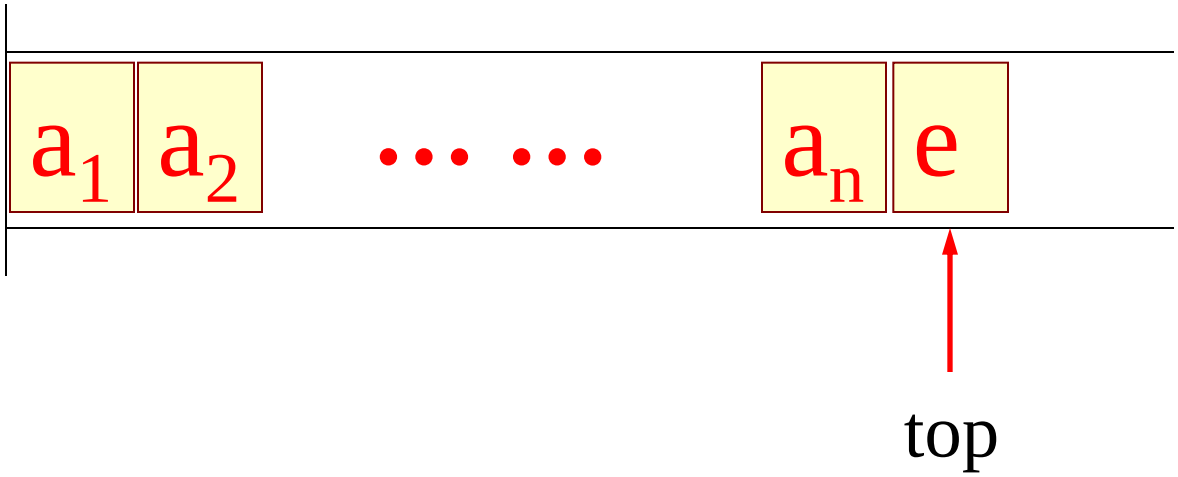
Basic Operations (5)

Push(&S, e)

Push Operation (Stack Insertion)

Condition: Stack S exists

Result: Insert element e as the new top



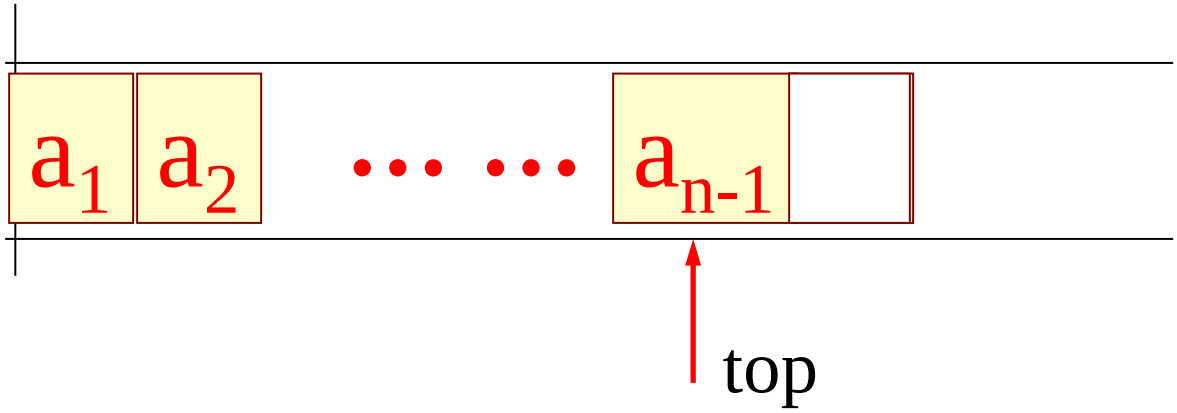
Basic Operations (5)

Pop(&S, &e)

Pop Operation (Stack Removal)

Condition: Stack S exists and is not empty.

Result: Remove the top element of S and return its value via e.



Example

Given the stack push order of three elements: 1, 2, 3. List all possible stack pop sequences of these three elements, and write the corresponding push and pop operation sequences (assuming "s" and "x" represent push and pop operations respectively).

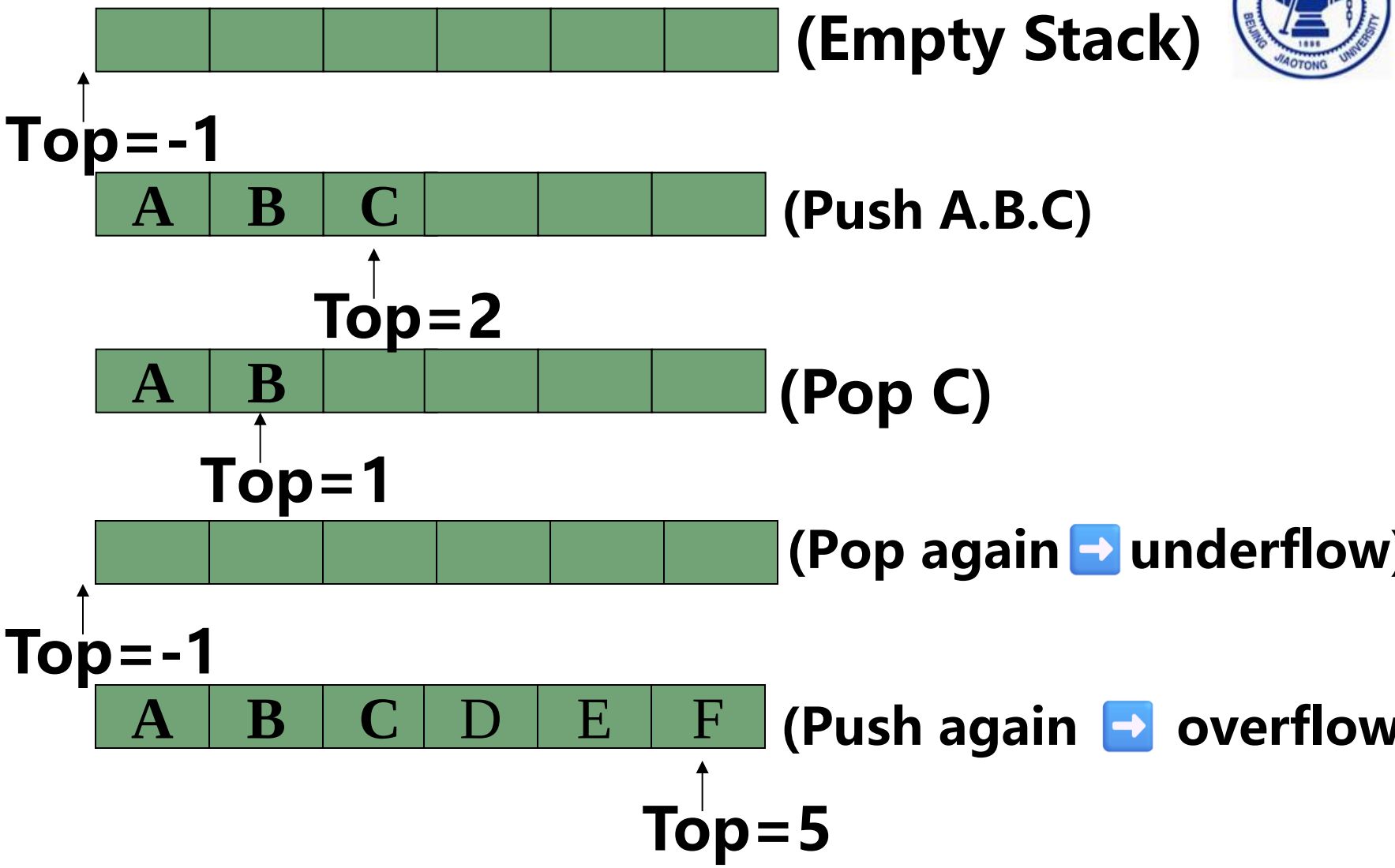
pop sequences	operation sequences
1、2、3	s x s x s x
1、3、2	s x s s x x
2、1、3	s s x x s x
2、3、1	s s x s x x
3、2、1	s s s x x x

Implementation of Stack Type

- **Sequential Storage Structure of Stack (Array)**
- **Linked Storage Structure (Link List)**

Sequential Storage Structure of Stack

- A set of consecutive memory cells (an array) is used to store several data elements sequentially from the bottom to the top of the stack.
- The upper bound of the array, maxsize, represents the maximum capacity of the stack.
- A top pointer, Top, is set to indicate the position of the top element in the arry.
- The bottom position of a stack is fixed, while the top position changes with push and pop operations.


(Empty Stack)
Top=-1
A B C
(Push A.B.C)
Top=2
A B
(Pop C)
Top=1
(Pop again ➡ underflow)
Top=-1
A B C D E F
(Push again ➡ overflow
Top=5

Define a Stack

```
#define  MAXSIZE   n
    /*n is the maximum possible number
of data elements in the stack*/
typedef struct {
  elemtype stack[MAXSIZE];
  int  top;
} sqstack;
```

Initialization operation

```
void Initstack(sqstack &s)
  {
   s . top= -1;    //Indicating the stack is empty


   }
```

Push operation

```
status push(sqstack &s, elemtype x)
    { if (s.top>=MAXSIZE-1)
            return ERROR;        //full stack，overflow
      else { s.top++;
             s.stack[top]=x;
             return OK;
           }
    }  //push
```

Pop operation

```
status pop(sqstack &s, elemtype &e)
    { if (s. top<0)
            return ERROR;        //empty stack, underflow
      else {  e= s . stack[top];
              s.top--;
              return OK;
            }
    }  // pop
```

Get the top element

```
status gettop(sqstack s, elemtype &e)
  { if (s . top<0)
        return NULL;        //empty stack
    else {
          e= s . stack[top];
          return OK;
        }
  }  // gettop
```

Stack Empty Judgment

```
status empty (sqstack s)
    { if (s . top<0)
            return TRUE;        //empty stack
      else {
              return FALSE;
            }
    }  // empty
```

When implementing these operations, the stack S does not need to be defined as a structure type (sqstack s); instead, it can also be defined as a pointer type pointing to the structure:

sqstack *s;

In this case:

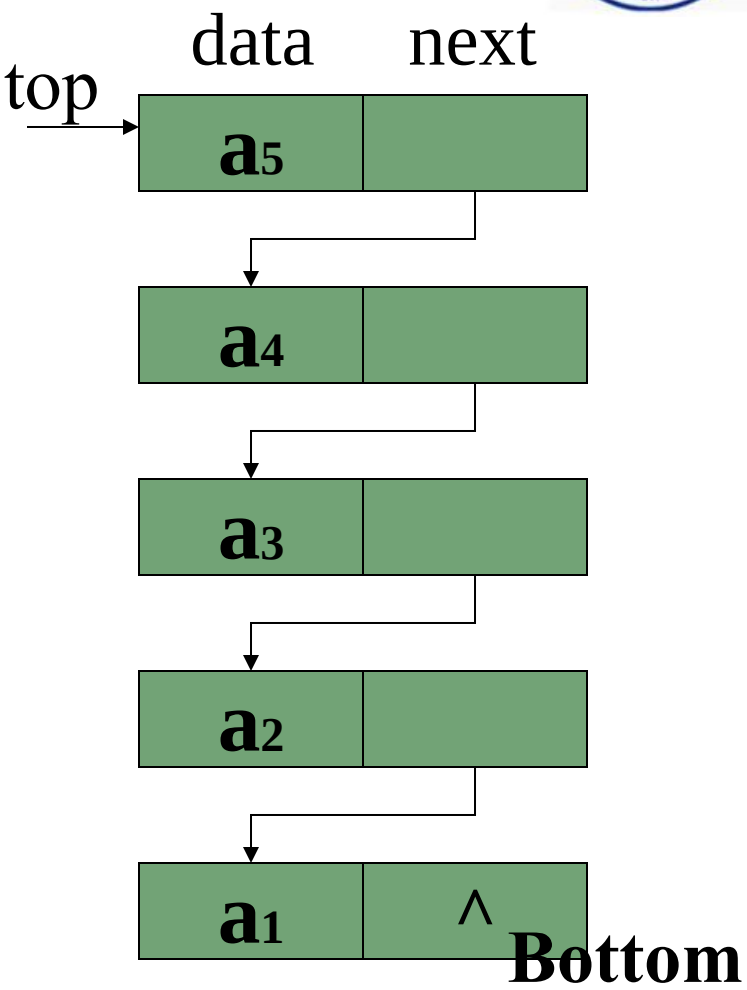
s .top change to s →top

s .stack[ ] change to s →stack[ ]。

Linked Storage Structure of Stack

When the maximum required capacity cannot be estimated in advance, using a linked storage structure is an effective method.

```
typedef struct  snode
    { elemtype  data;
      struct  snode  *next;
    }*linkstack;
```



Push Operation

```
status push (linkstack top, elemtype  x)
      {t= (linkstack) malloc (sizeof(snode));
       if  (t= =NULL)
          return ERROR;  //no enough memory space
       else  { t->data=x;   t->next=top;
                  top=t;
                  return OK; }
}//push
```

Pop operation

```
status pop  (linkstack top , elemtype  &e)
    { if  (top= = NULL)      //empty stack
                  return NULL;
            else  {linkstack p=top;
                    top=top->next;
                    e=p->data;
                    free (p);
                    return OK; }
    }//pop
```

Examples of Stack Applications

Example 1: Numbering System Conversion

Example 2: Checking Bracket Matching

Example 3: Line Editor Program Problem

Example 4: Maze Solving

Example 5: Expression Evaluation

Example 6: Implementing Recursion

Example 1: Numbering System Conversion

Based on:

$$N = (N \text{ div } d) \times d + N \bmod d$$

Example:

$(1348)_{10} = (2504)_8$,

Calculation

N	N div 8	N mod 8
1348	168	4
168	21	0
21	2	5
2	0	2

Output

```
void conversion ( ) {
    InitStack(S);
    scanf ("%d",N);
    while (N) {                    //N≠0
            Push(S, N % 8);
            N = N/8;
           }

   while (!StackEmpty(S)) {
           Pop(S,e);
           printf ( "%d", e );
           }
} // conversion
```

Complete Program

```
#define stacksize 100;
typedef struct
  { int base[stacksize];
    int top;
  } stack;
```

```
int push(stack *s,  int e)
{
  if (s->top>=stacksize-1)  return 0;
  s->top++; s->base[s->top]=e;   return 1;
}
int pop(stack *s,  int *e)
{
  if(s->top < 0)  return 0;
  *e=s->base[s->top]; s->top--;   return 1;
}
```

```
main()
{
  stack s1;     int m,e,n;
  s1.top=-1; m=1348; n=8;
  while(m)
   {  push(&s1, m%n ) ;     m=m/n;  }
  while(s1.top!=-1)
   {  pop(&s1,&e);  printf("%d",e);}
}
```

Example 2: Checking Brackets Matching

In an expression,

([] ()) or [([ ] [ ])]

is correct matching,

[( ]) or ([( )) or (()])

is incorrect matching.

The method for checking bracket matching can be described using the concept of "urgency of expectation"

Example：

[	(	[	]	[	]	)	]
1	2	3	4	5	6	7	8

Consider all the mismatching situations:

- The incoming right bracket is not the one that is "expected".

- What arrives is an "uninvited guest".
- Even until the end, the "expected" bracket never arrives.

The Design Idea

1) Whenever a left parenthesis appears, push it onto the stack.

2)Whenever a right parenthesis appears, first check if the stack is empty.

If the stack is empty, it indicates that this "right parenthesis" is redundant.

Otherwise, compare it with the top element of the stack

If they match, pop the left parenthesis.

Otherwise, it indicates a mismatch.

3)When the expression checking ends:

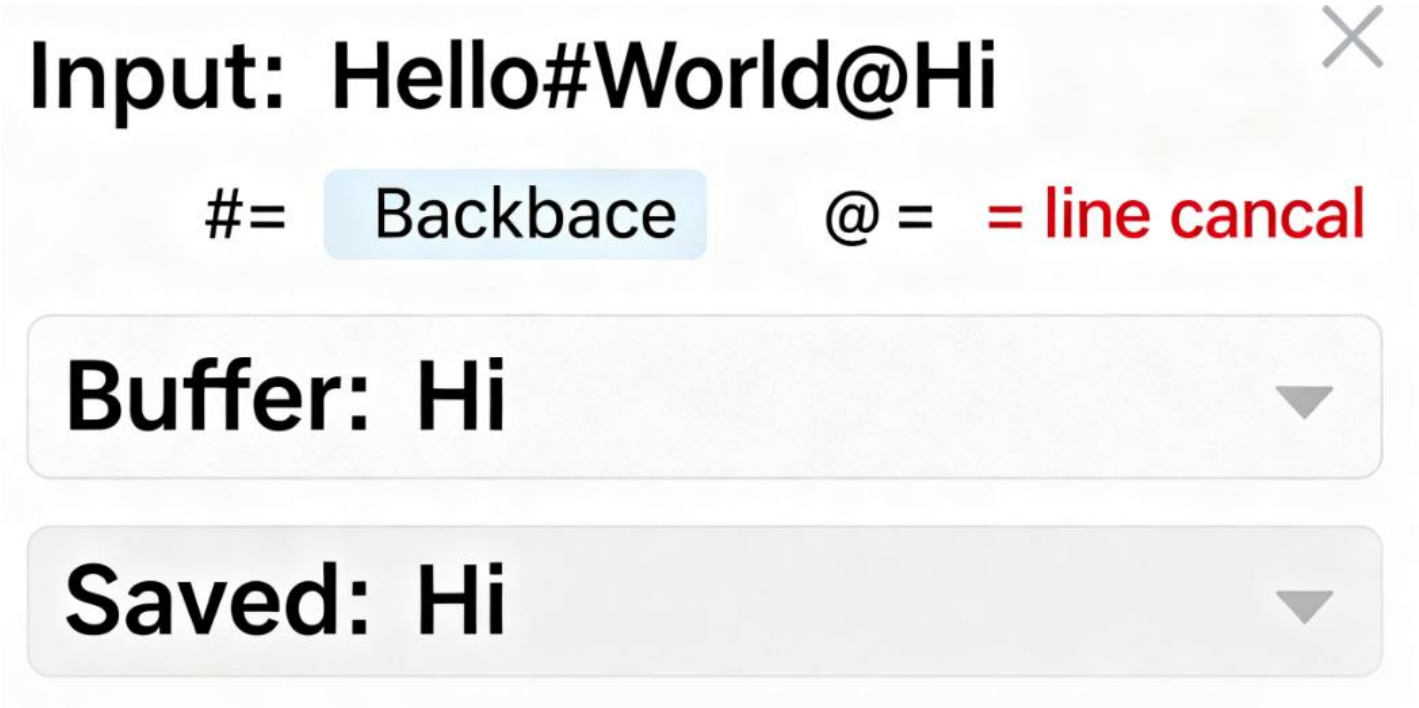
If the stack is empty, it indicates that the parentheses in the expression are matched correctly.

Otherwise, it indicates that there are extra "left parentheses".

```
Status matching(string& exp) {//Consider parentheses only
    int state = 1;  i=0;
    while (i<=Length(exp)-1 && state) {
        switch (exp[i]) {
         case "(":{Push(S,exp[i]); i++; break;}
         case ")": {
            if(NOT stackEmpty(S)&&GetTop(S)="(" )
                {Pop(S,e); i++;}
              else state = 0; break;} //mismatch
         default:  { state = 0; break; } }
    if (StackEmpty(S)&&state) return OK;
        else return ERROR
}//matching
```

Example 3：Line Editor Program Problem

The function of a simple line editor program is to receive data input by the user from the terminal and store them in the user's data area.



How to implement it?

"Store each character in memory as soon as it is received"?

Because users cannot guarantee that there will be no errors when inputting on the terminal, therefore, in the editing program, the approach of "storing each received character in the user data area immediately" is obviously not the most appropriate.

Implementation:

Set up an input buffer to receive a line of characters entered by the user, and then store them line by line into the user data area. Assume that "#" is the backspace character, and "@" is the line cancellation character.

Suppose the following two lines of characters are received from the terminal:

```
whli##ilr#e  (s#*s)
    outcha@putchar(*s=#++);
```

Then the following two lines are actually valid:

```
while (*s)
    putchar(*s++);
```

Use a **stack structure** to implement an input buffer for a line editor program：

- If the character received is **neither a backspace nor a line cancellation**, **push** this character onto the top of the stack；
- If the received character is a **backspace**, **pop** one character from the top of the stack；
- If the received character is a **line cancellation**, empty the stack.

```
void  LineEdit( )
   { InitStack(S);                          //Initialize an empty stack
     ch=getchar( );                         //Receive a character
     while (ch!=EOF)                        //EOF is the end mark
       {  while (ch!=EOF && ch!='\n')
          { switch (ch)
              { case '#': Pop(S,ch);      break;
                case '@':ClearStack(S);break; //Empty stack S
                default:  Push(S,ch); break;
              }
              ch=getchar( );       // Receive next character
          }
        …
        Transfer the characters in the stack to the data area;
        …
        ClearStack (S);                     //Reset S to an empty stack
        if  (ch!=EOF)  ch=getchar( );
       }
     DestroyStack(S);
}//LineEdit
```

Example 4: Maze Solving

Start from the right direction of the point and rotate clockwise

"exhaustive solution" method

entrance

#	#	#	#	#	#	#	#	#	#
#	→	→	#	$	$	$	#		#
#		↓	#	$	$	$	#		#
#	↓	←	$	$	#	#			#
#	↓	#	#	#				#	#
#	→	→	↓	#				#	#
#		#	→	→	↓	#			#
#	#	#	#	#	↓	#	#		#
#					→	→	→	Θ	#
#	#	#	#	#	#	#	#	#	#

The basic idea of the maze path - finding algorithm:

- If the current position is "passable", it is included in the path and we continue to move forward;
- If the current position is "impassable", it is removed from the path, we go back and change direction to continue exploring.

Maze path - finding algorithm

Let the current position to be the entrance;

do{

if current position is passable,

then{**push** current position to the stack**;**

if this is the exit, then the algorithm ends;

otherwise go to its **right** position as current position;

}

otherwise {

A:… …

}

}while (the stack is not empty）;

A:

if the stack is not empty and the top of the stack has other directions to be explored,

{then set the new current position to be: the next adjacent block of the top position by rotating clockwise;}

if the stack is not empty and the top of the stack is impassable,

{then pop the top position out;

if the stack is not empty, test the next top;

until a passable position is found or the stack is empty;

}

if the stack is empty, then there is no pass in the maze.

Example 5: Evaluating expressions

- Evaluating expressions is a fundamental problem in the compilation of programming languages, and its implementation is another typical example of stack application.

- Here, we introduce a simple, intuitive, and widely used algorithm called the "operator precedence method". It realizes the compilation or interpretive execution of expressions according to the regulations of operation precedence relations.

Three notation methods for expressions:

Let Exp = S1 + **OP** + S2

then S1 + **OP** + S2 **infix notation**

OP + S1 + S2 **prefix notation**

S1 + S2 + **OP** **postfix notation**

Example: **Exp** = $\underline{a \times b} + \underline{(c - d / e) \times f}$

prefix: $+ \ \underline{\times \ a \ b} \ \underline{\times - c / d \ e \ f}$

infix: $\underline{a \times b} + \underline{c - d / e \times f}$

postfix: $\underline{a \ b \times} \ \underline{c \ d \ e / - f \times} +$

1) The relative order between operands remains unchanged;

2) The relative order of operators is different;

3) The infix expression loses the parenthesis information, resulting in an uncertain order of operations.;

4)The prefix operation rule is:

The two consecutive operands and the operator preceding them and immediately adjacent to them form a minimal expression;

5)The postfix rules of operation are:

Each operator and the two operands that appear before it and immediately adjacent to it form a minimum expression; the order in which the operator appears in the expression is exactly the order of operation of the expression.

- **How to get the suffix from the original expression?**

Analyze the operators in "expression" and "postfix":

Original expression: **a + b × c – d / e × f**

Postfix: **a b c × + d e / f × –**

The order of each operator is determined by the next operator after it, and in postfix, operators with higher priority precede those with lower priority.

The rule for obtaining suffix from the original expression is:

1) Set up the operator stack;

2) The expression is terminated by "#",

The bottom of the preset operator stack is "#";

3) If the current character is an operand,

Send it directly to the suffix.

The rule for obtaining the suffix from the original expression is:

4) If the priority number of the current operator is higher than that of the top operator, enter the stack;

5) Otherwise, the operator at the top of the exit stack is sent to the suffix;

6) ")" Acts as an isolation of the operator before and after it, and "") can be regarded as the end of an expression starting from the corresponding left bracket.

Example: Exp = a × b + (c – d / e) × f#

Postfix: Suffix = a b × c d e / – f × +

#

operator stack

```
void transform(char suffix[ ], char exp[ ] )
        // Derive the postfix from the original expression
        // S is the operator stack, s is preset with # at the bottom of the stack,
        // and OP is the set of operators
{    InitStack(S);  Push(S, '#');     p = exp;  ch = *p;
     while (!StackEmpty(S))
      { if (!IN(ch, OP))                    // If ch is an operand
            Pass( Suffix, ch);
      else {   A:        … ..}  // If ch is an operator
         if ( ch!= '#' ) { p++;  ch = *p; }
            else { Pop(S, ch);  Pass(Suffix, ch); }
      } // while
} // CrtExptree
```

A:

```
switch (ch)
 { case '(' :  Push(S, ch); break;
   case ')' :  Pop(S, c);
               while (c!= '(' )
                  { Pass( Suffix, c);  Pop(S, c) }
               break;
   defult  :
      while(Gettop(S, c) && ( precede(c,ch)))
        { Pass( Suffix, c);  Pop(S, c); }
      if ( ch!= '#' )  Push( S, ch);
      break;
} // switch
```

If the priority of ch is lower than that of c, it is true

expression
- **operand**： 1,2,x,y
- **operator**： +-*/
- **delimiter**： ( )

Two stacks can be used:

stack OPTR：used to store operators

stack OPND：used to store operands or operation results

Basic idea:

(1) Set the OPND stack as an empty stack, and let the expression start symbol "#" be the bottom element of the OPTR stack;

(2) Read each character in the expression in sequence. If it is an operand, push it into the OPND stack; if it is an operator, compare its priority with the operator at the top of the OPTR stack and perform corresponding operations until the entire expression is evaluated (that is, both the top element of the OPTR stack and the currently read character are "#").

```
OperandType EvaluateExpression( )
   { InitStack(OPTR); Push (OPTR,'#');
      InitStack(OPND); c=getchar( );
     while (c!='#' || GetTop(OPTR))!='#')
        { if (!In(c,OP)          //c is not a operator
              { Push ((OPND,c);c=getchar( );} //push operand
        else  switch (Precede(GetTop(OPTR),c)
                { case '<' : Push(OPTR,c); c=getchar( );
                        break; // The top element of the stack has low
                                //priority and operators are pushed onto the stack
                  case '=' : Pop(OPTR,x); c=getchar( );
                              break; // Remove parentheses and receive
                                     //the next character
                  case '>': Pop(OPTR, theta);
                              Pop(OPND,b); Pop(OPND,a);
                              Push(OPND,Operate(a,theta,b));
                              break; //pop the stack and push the result onto the stack
                  } //switch
        } //while
         return GetTop(OPND);
   } // EvaluateExpression
```

How to evaluate by suffix?

Start with the operator,
Find the operand again

For instance：

a b × c d e / − f × +

a×b

d/e

c-d/e

(c-d/e)×f

To get the value of a suffix:

1) Set up the operand stack S;

2) The expression is terminated by "#";

3) Read a character of the expression. If the current character is an operand, push it into the stack and go to 4).

4) If the current character is an operator optr, two numbers are popped from the stack and the result of the operation is pushed into the stack, i.e.:

x2=POP(S)； x1=POP(S)；

PUSH(S,(x1 optr x2));

5) Read the next character and repeat 3) and 4) until you read the end symbol #;

6) At the top of the stack, x=POP (S) is the value of the corresponding expression in postfix notation.

```
int  cal(char suffix-exp[ ] )
// Calculate the value of the postfix expression
//s is the stack of operations, OP is the set of operators
 {
     InitStack(S); i = 0;  ch = suffix-exp[0];
     while (ch<>'#')
         { if (!IN(ch, OP))            //  If ch is an operand
             Push( S, ch);
           else                        // if ch is an operator
         { x2=POP(S);   x1=POP(S);  //Take two operands
                PUSH(S,(x1 ch x2));
             }
           i++;  ch= suffix-exp[i];
         } // while
 } //cal
```

Example 6 Nesting and Recursion (1)

- **When one function calls another during the execution of a function, three tasks must be completed before the called function can be executed:**
  - **Pass all real parameters, return addresses and other information to the called function for preservation;**
  - **Allocate storage for local variables of the called function;**
  - **Transfer control to the entry point of the called function.**
- **NB.Three consistencies of function calls (number, order, type).**

Before returning from the called function, the following three tasks should be completed:

- **Save the calculated results of the called function;**
- **Release the data area of the called function;**
- **The control is transferred to the calling function according to the return address stored in the called function.**

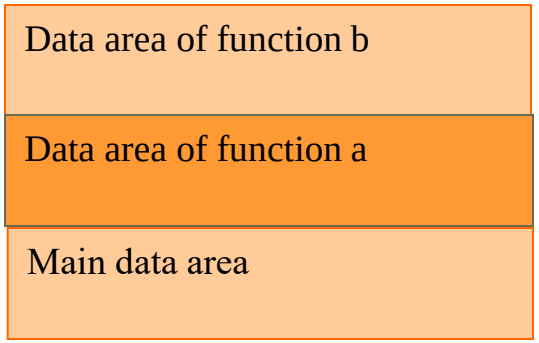
The rules for nested function calls are:

The latter call returns first!

Memory management at this time is "stack management"

for instance：

```
void main( ){    void  a( ){      void  b( ){
…                …                …
a( );            b( );
…                …
}//main          }// a           }// b
```



Example 6 Nesting and Recursion (4)

- **Function communication and control transfer must be managed through a stack. The system allocates the entire data space required for program execution within a single stack. Each function call reserves a storage area at the top of the stack, while each function exit releases its corresponding storage space. Consequently, the data area of the currently executing function resides at the stacks top.**

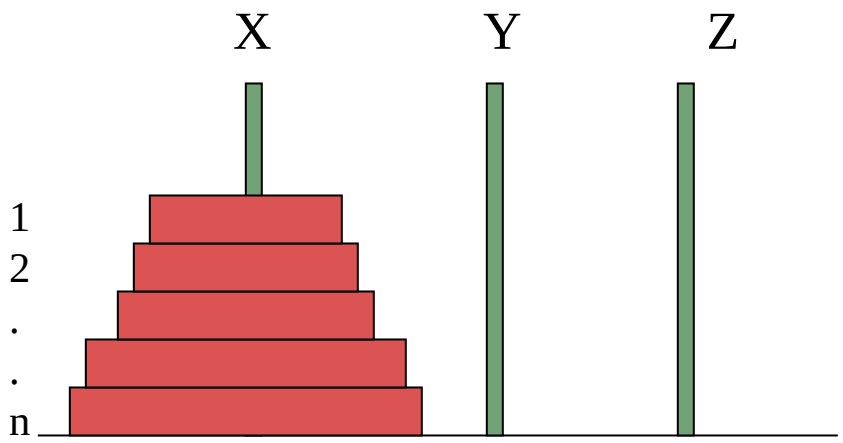
Example 6 Nesting and Recursion (5)

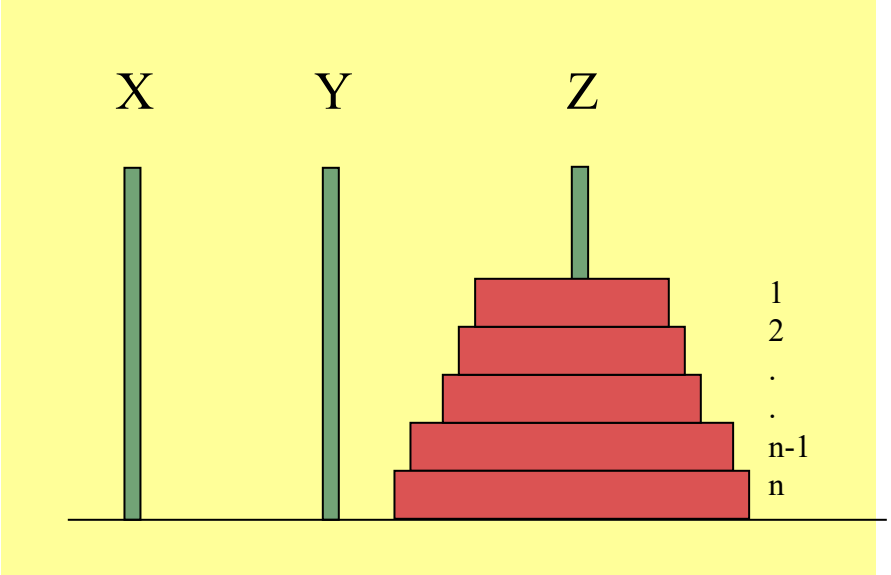
- **The operation of a recursive function resembles nested function calls, except that the calling and called functions are the same. Note the "hierarchical" nature of recursive function execution: The main function is at level 0, and each recursive call from the main function enters level 1.**

- **Each time a recursive layer is entered, a new work record (including the return address of the previous layer, the actual parameters, and local quantities) is created and pushed onto the stack. Each time a recursive layer is exited, a work record is popped from the stack.**

The n-level Tower of Hanoi problem:

Suppose there are three towers named X, Y, and Z. Tower X contains n disks of varying diameters, numbered 1, 2,..., n in ascending order. The goal is to move all these disks from Tower X to Tower Z while maintaining their original sequence.



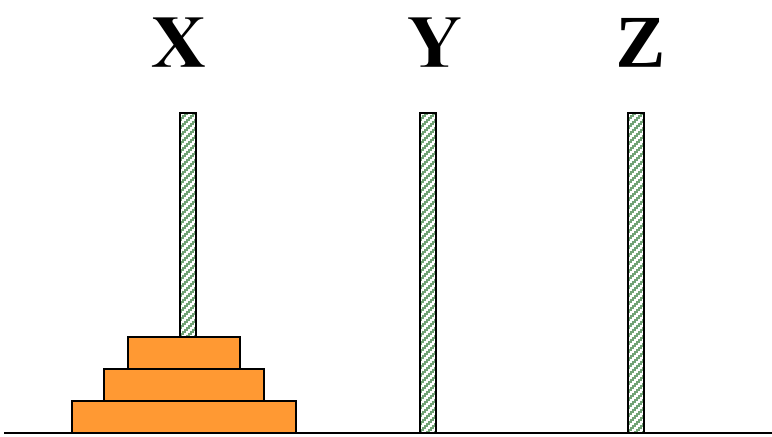


The following rules must be followed when the disk is moved:

1) Only one disk can be moved at a time;

2) The disc can be inserted into any tower in X, Y and Z;

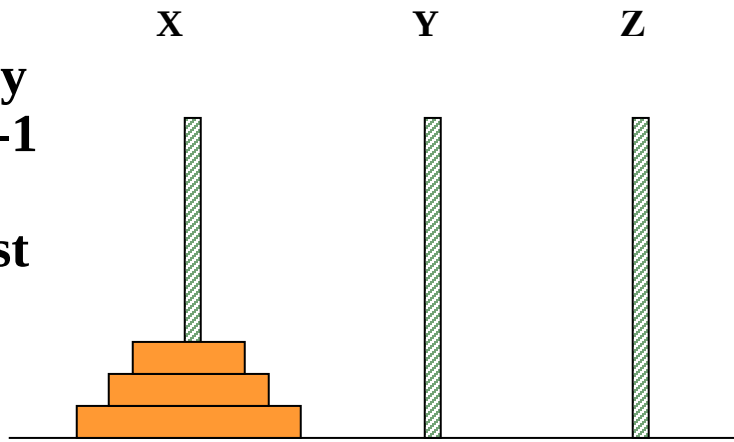
3) Never put a larger disc over a smaller one at any time.



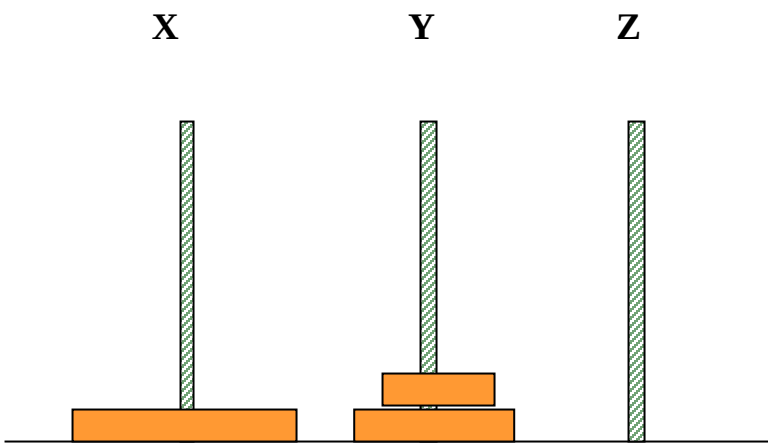
How to move the disk?

When n=1, from the tower seat X→Z;

When n>1, using the tower base Y as an auxiliary tower base, if we can find a way to remove the n-1 discs that are pressing on the disc numbered n from the tower base X,→On the top, you can first take the disc numbered n from the tower base X→Z, and then move n-1 disks from tower Y to tower Z.



The problem of moving n-1 disks from one tower to another is a problem with the same characteristics as the original problem, but with one smaller size, so it can be solved in the same way.



```
void hanoi (int n, char x, char y, char z)
// Number the tower base x from 1 to n in ascending order of diameter and from bottom to top
// The n disks are moved to the tower base z, and y can be used as an auxiliary tower base.
{
 if (n==1)
      move(x, 1, z);          // moves the disc numbered 1 from x to z
 else {
      hanoi(n-1, x, z, y);   // Assign numbers 1 to n-1 to x
                              // The disk moves to y,z as an auxiliary tower
      move(x, n, z);          // moves disc numbered n from x to z

      hanoi(n-1, y, x, z); // The numbers from 1 to n-1 on y
                              // The disc is moved to z, x as an auxiliary tower
        }
}
```

```
void hanoi (int n, char x, char y, char z)
  {
1  if (n==1)
2   move(x, 1, z);
3  else
4   {  hanoi(n-1, x, z, y);
5       move(x, n, z);
6       hanoi(n-1, y, x, z);
7   }
8 }
```

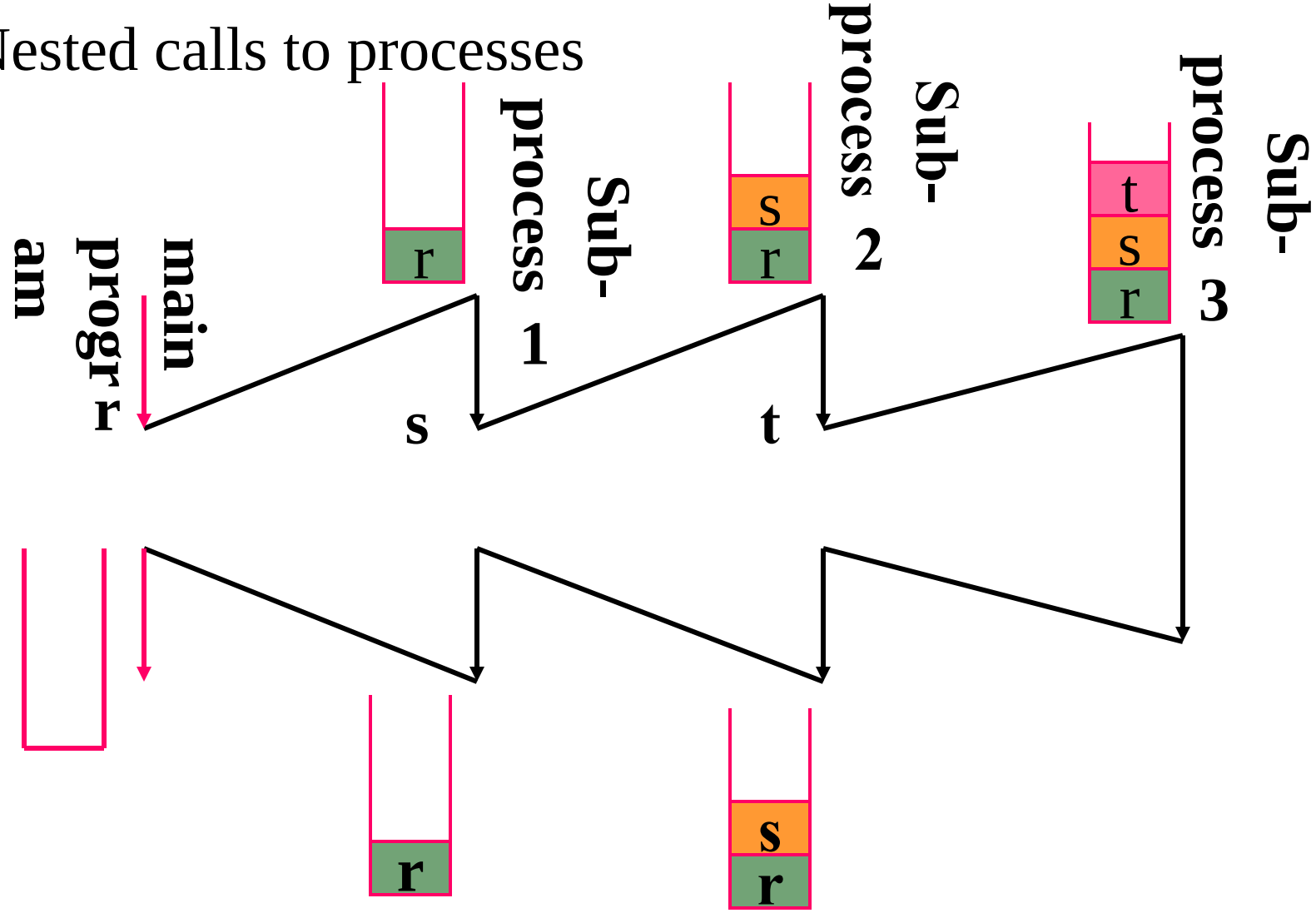
5	2	a	c	b
0	3	a	b	c

statement label

Return address n x y z

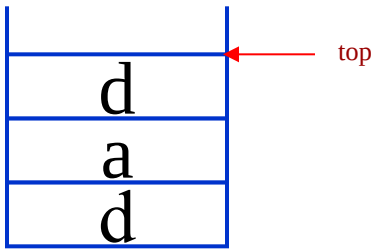
Stack applications

Nested calls to processes



Palindromic game:

Read the string forward and backward (without spaces)



String: "madam im adam"

Algorithm idea:

1.Read into string

2.Remove the space (original string)

3.Push into stack

4.The original string character is compared with the out stack character in sequence

If not equal, non-palindrome

If the stack is empty, its equal, and its a palindrome

int IsHuiwen(char *S)

```
{  SeqStack T;
   int i , n;   char t1;
   InitStack( &T);   n=strlen(S);//Length of the vector
   for ( i=0; i<n/2; i++)
         Push( &T,  S[i]);
      //Push half of the characters onto the stack
   i=n/2-1;
   while(i>=0)
     { t1=Pop (&T);
      //Compare each character that pops up with the corresponding
character
      if( t1!=S[n-i-1]) return 0 ;   //If not, return 0
        i--;
     }
    return  1 ;  // If the comparison is complete and equal, return 1

}
```

Map four staining problem

- **The four-color theorem is one of the famous theorems in computer science.**

- **Make adjacent countries or administrative regions on the map not use the same color, at least four colors can be used to color the map.**

- **To prove the conclusion of this theorem, we use a stack to color the map by backtracking.**

Map four staining problem

- **Ideas: Number of administrative districts: 1-7**

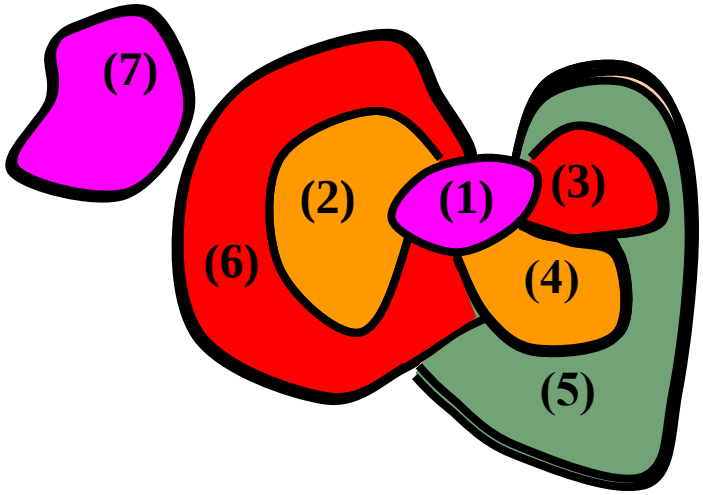
- **Color code: a, b, c, d;**

- **Starting from the first administrative zone, each area undergoes sequential color testing using four distinct hues. If a color matches adjacent zones without overlap, its numerical value is recorded in the stack. Should any color overlap occur between the current and adjacent zones (a-d), the system will pop the value from the top of the stack to correct the current color code.**

Example

R [7][7]

	1	2	3	4	5	6	7
1	0	1	1	1	1	1	0
2	1	0	0	0	0	1	0
3	1	0	0	1	1	0	0
4	1	0	1	0	1	1	0
5	1	0	1	1	0	1	0
6	1	1	0	1	1	0	0
7	0	0	0	0	0	0	0



1# Purple

2# Yellow

3# Red

4# Blue

S[K]

1	2	3	4	5	6	7
1	2	3	2	4	3	1

```
Void  mapcolor(int R[][],int n,int s[])
 {  s[1]=1;          // Region 1 is colored in one color
    a=2;  J=1;          //a is the region number, J is the chromosome number
    while ( a<=n)
    { while(( J<=4)&&(a<=n))
        { k=1;                // k represents the number of colored regions
          while(( k<a)&&(s[k]*R[a-1][k-1]!=J))       k=k+1;
              // If not adjacent, or if adjacent and non-color, judge the next area.
          IF (k<a)   J=J+1;        //adjacent and same color
          ELSE{ s[a]=J;  a=a+1;  J=1; }
            //adjacent and non-overlapping colors
         }
      IF (J>4) { a=a-1; J=s[a]+1  }
    }
 }
```

Solving the Queen Problem

The solution to the n-queens problem is (x 1, x 2, x 3,..., x n),

The constraint is that any two x i and x j cannot be in the same row, column or diagonal of the board.

How do you represent the pieces placed on the board?

Since there can only be one piece on each row, column, and diagonal, each column only needs to record the row number of the piece in that column, so a one-dimensional array is used.

An example of solving the four queens problem

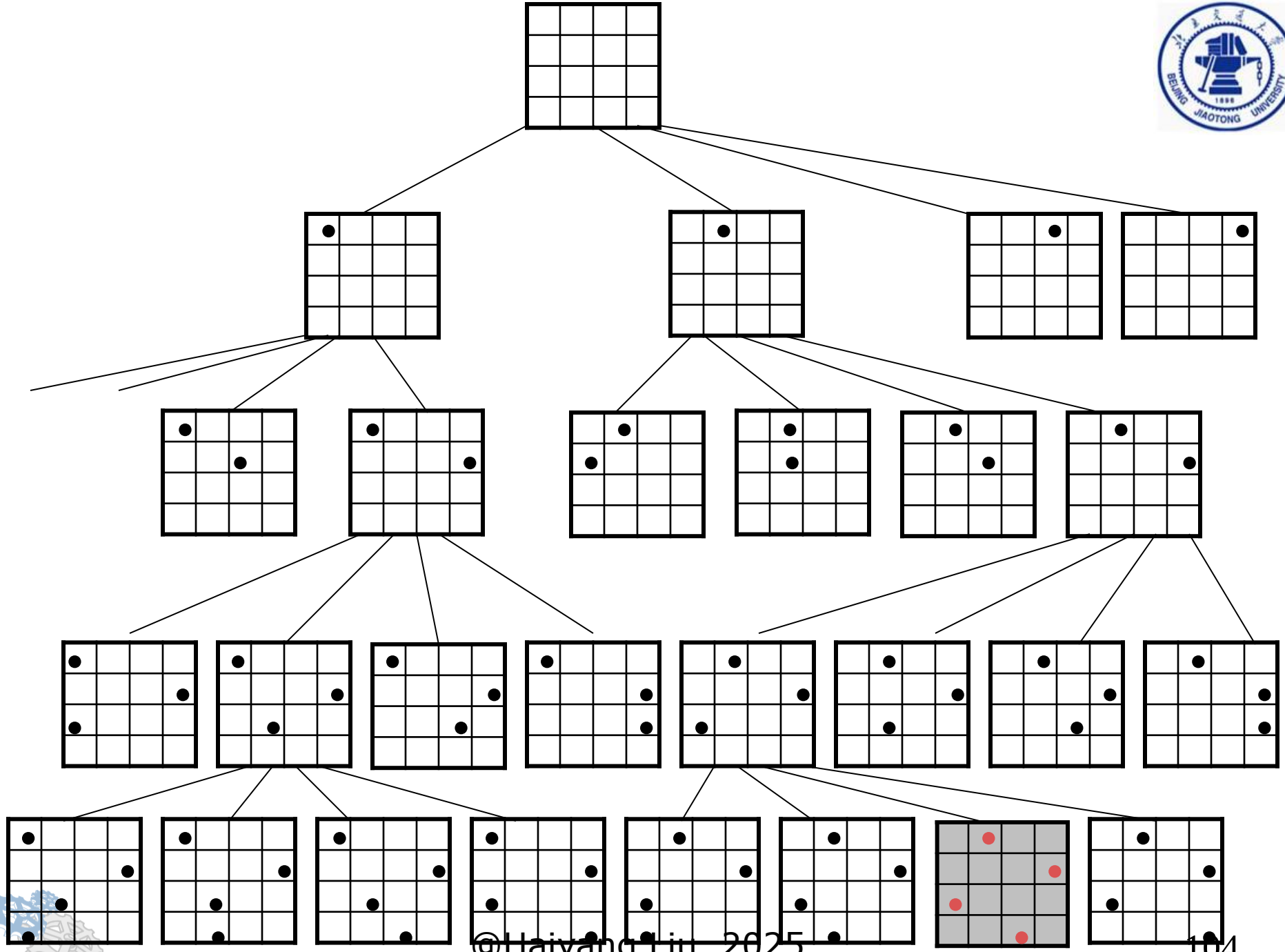
Let the solution to the four queens problem be (x 1, x 2, x 3, x 4),

Among them: x i (i=1,2,3,4) □ S i={1,2,3,4}

The constraint is that any two x i and x j cannot be in the same row, column and diagonal of the board.

According to the definition of backtracking, the solution process of the queen problem is:

The initial value of the solution is empty. First, add x_1=1, then add x_2=3 that satisfies the condition. Since no partial solution (x_1, x_2, x_3) satisfying the constraint can be found for all x_3 □ {1,2,3,4}, backtrack to the partial solution (x_1), and re-add x_2=4 that satisfies the constraint. This process continues sequentially (by row and column index).



```
void queen(int i, int n)
{ // When entering this function, the previous i-1 rows of the n×n board have been placed without
attacking each other
// The i-th piece is clicked. Now continue to select for the subsequent pieces starting from the i-th row.
// The position that satisfies the constraints. When a valid layout is obtained for (i> n)
// When output is produced.
if (i>n) outputs the current layout of the board;
  else  for (j=1;  j<=n; ++j)
        {   Place a chess piece in row i and column j;
            if (current layout is legal) queen(i+1, n);
            Remove the pieces in row i and column j;
        }
} // trial
```

```
#include<stdio.h>
#define n 8 // n is the number of queens, m is the number of moves
int m=0, a[n]; // a[i] stores the row number where the i-th queen is
placed,
int ok (int i, int j) // Check whether the piece can be placed on (i, j)
{j1=j; i1=i; ok1=1; // Check if a piece can be placed on line i
while( (j1>1)&&ok1)
{j1--;        ok1=a[j1]!=i ;}
j1=j; i1=i; // Check whether a piece can be placed on the diagonal
while( (j1>1)&&(i1>1)&&ok1)
{j1--; i1--; ok1=a[j1]!=i1 ;}
j1=j; i1=i; // Check whether a piece can be placed on the other
diagonal
while( (j1>1)&&(i1<n)&&ok1)
{j1--; i1++; ok1=a[j1]!=i1 ;}
return  ok1
}
```

```
Void  queen(int j) //Start at column j and test one by one
{  if (j>n)
     { m++;  printf("m=%d   ",m);
       for (i=1;i<=n;i++)  printf("    %d",a[i]);
       printf( "\n" );
      }
   else  for( i=1; i<=n;i++)
           if(ok(i,j))  //Check whether a piece can be placed on (i, j)
             { a[j]=i; // place a piece on (i, j)
               queen(j+1) ;
              }
}
main()
{queen(1);}
```

Thank You !

Haiyang Liu

Email: haiyangliu@bjtu.edu.cn

School of Computer & Information Technology, Beijing Jiaotong University, Beijing, 100044, China